2026 春

程序设计实习 第四讲

继承与派生

杨 帅
williamyang@pku.edu.cn

上节回顾

- 运算符重载基本概念
- 赋值运算符的重载
- 运算符重载为友元函数
- 流插入运算符和流提取运算符的重载
- 类型转换运算符的重载
- 自增、自减运算符的重载

上节回顾

- **三种运算符重载的实现方式**
  - 重载为普通函数
  - 重载为成员函数
  - 重载为友元
- **常见的各种运算符重载**
  - 流运算符(>>/<<)
  - 自增/自减运算符(++/--)
  - 赋值号(=)
  - 下标运算符([])
  - 类型转换运算符

测一测

- **以下关于赋值运算符重载的说法，正确的是：**
  - 赋值运算符重载成全局函数时，应该有两个参数
  - 赋值运算符重载的唯一目的就是使得可以用其他类型的变量或常量给对象复制
  - 赋值运算符重载时，返回值设为其所作用的对象的引用，是符合赋值运算符使用习惯的做法
  - 赋值运算符重载时，返回值设为void是符合赋值运算符使用习惯的做法

测一测

- **以下关于赋值运算符重载的说法，正确的是：**
  - 赋值运算符重载成全局函数时，应该有两个参数
  - 赋值运算符重载的唯一目的就是使得可以用其他类型的变量或常量给对象复制
  - **赋值运算符重载时，返回值设为其所作用的对象的引用，是符合赋值运算符使用习惯的做法**
  - 赋值运算符重载时，返回值设为void是符合赋值运算符使用习惯的做法

测一测

- 如果将运算符 * 重载为某个类的成员运算符（也即成员函数），则该成员函数的参数个数是：
  - 0个
  - 1个
  - 2个
  - 0个1个均可

测一测

- 如果将运算符 * 重载为某个类的成员运算符（也即成员函数），则该成员函数的参数个数是：
  - 0个
  - 1个
  - 2个
  - 0个1个均可

测一测

程序输出结果如下，请填空

```
0
5
```

```
class A {
public:
    int val;
    A(____________ ){    val = n;    }
    ___________ GetObj() {
        return ________;
    }
};
```

```
int main(){
    A a;
    cout <<a.val << endl;
    a.GetObj() = 5;
    cout << a.val << endl;
    return 0;
}
```

测一测

程序输出结果如下，请填空

```
0
5
```

```
class A {
public:
    int val;
    A(int n=0){    val = n;    }
    A & GetObj() {
        return *this;
    }
};
```

```
int main(){
    A a;
    cout <<a.val << endl;
    a.GetObj() = 5;
    cout << a.val << endl;
    return 0;
}
```

测一测

程序输出结果如下，请填空

```
0
5
```

```
class A {
public:
    int val;
    A(int n=0){    val = n;    }
    int & GetObj() {
        return val;
    }
};
```

```
int main(){
    A a;
    cout <<a.val << endl;
    a.GetObj() = 5;
    cout << a.val << endl;
    return 0;
}
```

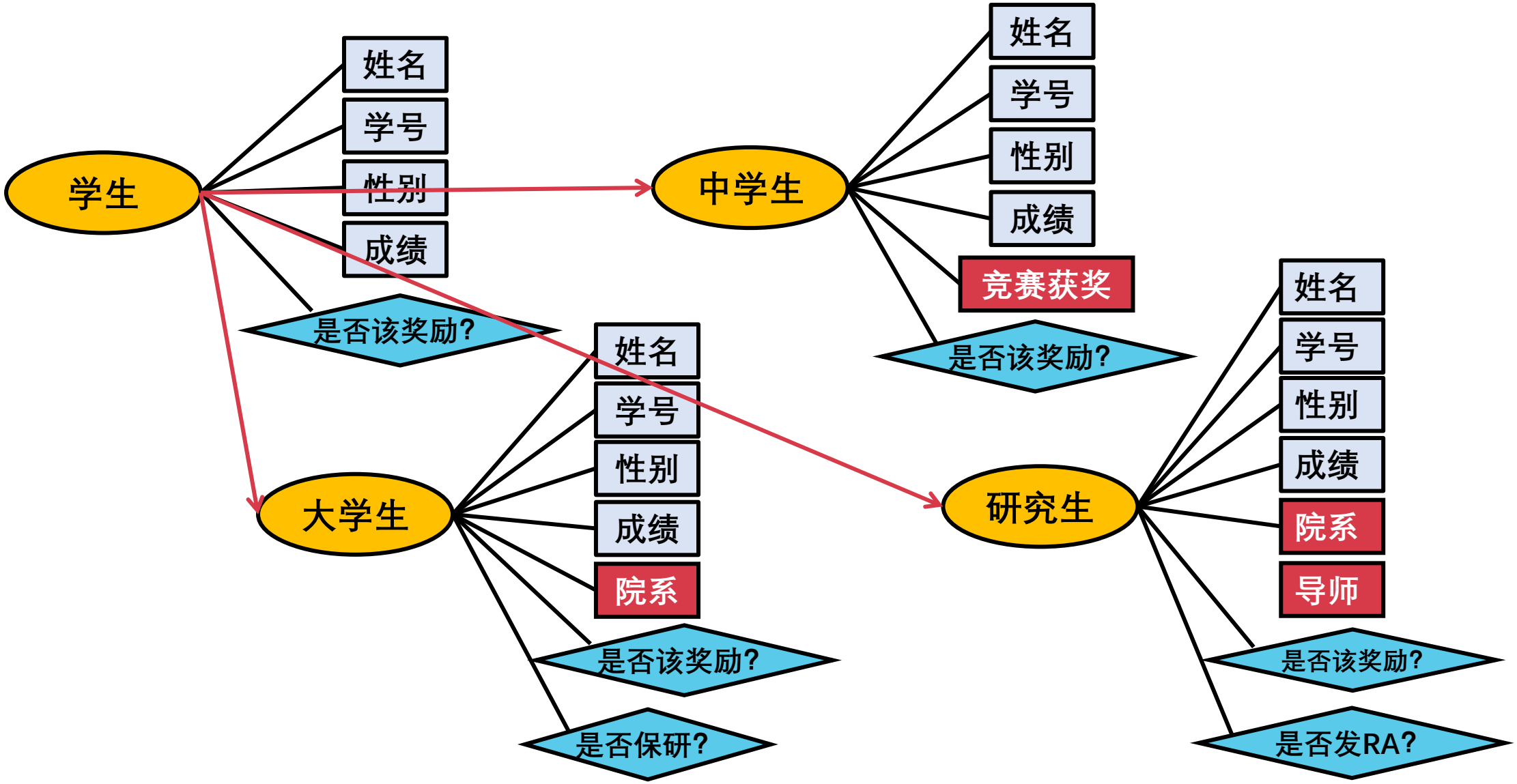
目　录

- 基本概念：继承，基类，派生类
- 派生类的成员组成，可见性
- 派生类的构造，析构
- 派生类与基类的指针类型转换

目　录

- 基本概念：继承，基类，派生类
- 派生类的成员组成，可见性
- 派生类的构造，析构
- 派生类与基类的指针类型转换

继承机制：从例子开始


学生
姓名
学号
性别
成绩
是否该奖励?
中学生
姓名
学号
性别
成绩
竞赛获奖
是否该奖励?
大学生
姓名
学号
性别
成绩
院系
是否该奖励?
是否保研?
研究生
姓名
学号
性别
成绩
院系
导师
是否该奖励?
是否发RA?

继承与派生的概念

- **继承**
  - 在定义一个新的类B时，如果该类与某个已有的类A相似
  - **（指的是B拥有A的全部特点）**
- A–**基类（父类）**，B–基类的**派生类（子类）**
  - **派生类**是通过对基类进行**修改**和**扩充**得到的
    - **扩充**：在派生类中，可以**添加**新的成员变量和成员函数
    - **修改**：在派生类中，可以**重新编写**从基类继承得到的成员
  - 派生类一经定义后，可以独立使用，不依赖于基类

继承与派生的概念

- **继承**
  - C++中，从一个类派生出另一个类的写法：

```
class 派生类名 : 派生方式说明符 基类名 {
    ...
}
```

  - **派生类拥有基类的全部成员，包括**
    - private/protected/public成员变量
    - private/protected/public方法
  - **在派生类的各个成员函数中，不能访问基类中的private成员**

需要继承机制的例子

- 所有的学生都有一些**共同属性和方法**
  - **属性**：姓名，学号，性别，成绩等
  - **方法**：判断是否该留级，判断是否该奖励等
- 不同的学生，比如中学生/大学生/研究生，又有各自**不同的属性和方法**，例如
  - 大学生有系的属性，而中学生没有
  - 研究生有导师的属性
  - 中学生竞赛、特长加分之类的属性

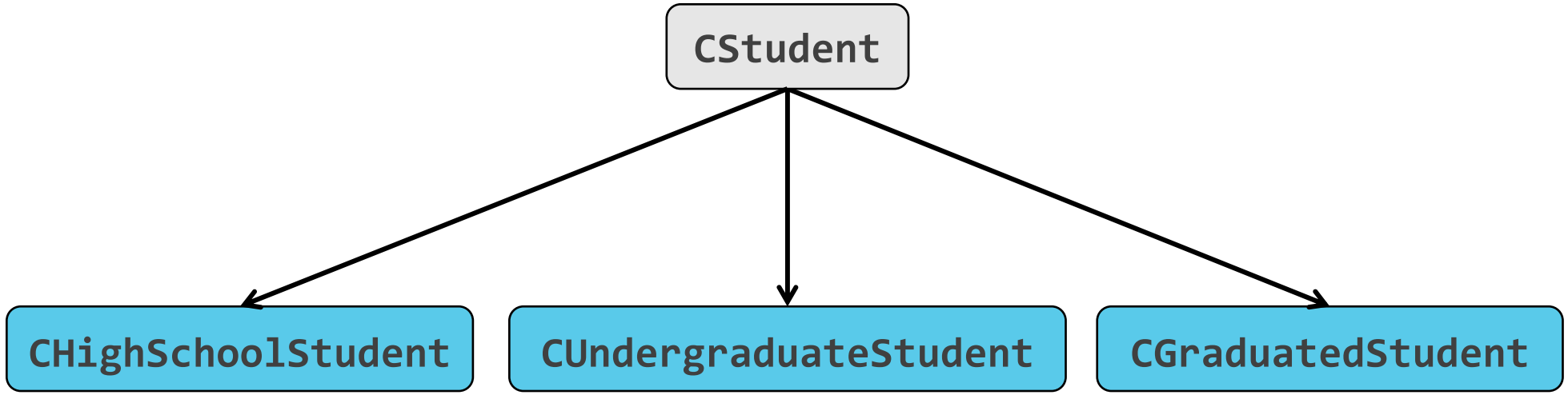
需要继承机制的例子

- 如果为每类学生都编写一个类

  → 不少重复的代码，浪费

- **比较好的做法：**
  - 编写一个 "学生" 类，概括了各种学生的共同特点
  - 从"学生"类派生出"大学生"类，"中学生"类，"研究生类"

需要继承机制的例子


CStudent
CHighSchoolStudent
CUndergraduateStudent
CGraduatedStudent

需要继承机制的例子


```
class CStudent {
private:
    string sName;
    int nAge;
public:
    bool IsThreeGood() { }
    void SetName( const string & name )
    {    sName = name;    }
    // ...
};
```

需要继承机制的例子


```
// 派生类的写法是：类名：派生方式说明符 基类名
class CUndergraduateStudent : public CStudent {
private:
    int nDepartment;
public:
    bool IsThreeGood() {  ...  }  // 派生类同名函数隐藏基类函数
    bool CanBaoYan() { ... }
};
```

需要继承机制的例子


```
class CGraduateStudent : public CStudent {
private:
    int nDepartment;
    char szSupervisorName[20];
public:
    int CountSalary() { ... }
};
```

派生类对象的内存空间

- 派生类对象的大小，等于 **基类对象的大小 + 派生类对象自己的成员变量的大小**
- 在**派生类对象**中，包含着基类对象，而且基类对象的存储位置位于派生类对象新增的成员变量之前

派生类对象的内存空间

- 派生类对象的大小，等于 **基类对象的大小 + 派生类对象自己的成员变量的大小**
- 在**派生类对象**中，包含着基类对象，而且基类对象的存储位置位于派生类对象新增的成员变量之前

```
class CBase {
    int v1,v2;
};
class CDerived: public CBase {
    int v3;
}
```

派生类对象的内存空间

- 派生类对象的大小，等于 **基类对象的大小 + 派生类对象自己的成员变量的大小**
- 在**派生类对象**中，包含着基类对象，而且基类对象的存储位置位于派生类对象新增的成员变量之前

```
class CBase {
    int v1,v2;
};
class CDerived: public CBase {
    int v3;
}
```

复合与继承

- **继承："是" 关系**
  - 基类 A
  - B是基类A的派生类
  - 逻辑上要求："一个B对象也是一个A对象"
- **复合："有" 关系**
  - 复合，即一个类的对象拥有作为其成员的其它类的对象
  - d是类C的成员
  - d是类D的一个对象
  - 复合关系满足：C类中 "有" 成员对象d

继承的使用

- 写了一个 CMan 类代表男人 → 又发现需要一个 Cwoman 类来代表女人
  - 仅仅因为CWoman类和CMan类有共同之处，就让CWoman类从CMan类派生而来，是不合理的
  - 因为 "一个女人也是一个男人" 从逻辑上不成立
- 好的做法是概括男人和女人共同特点，写一个 CHuman 类，代表 "人"，然后CMan和CWoman都从CHuman派生

复合关系的使用

- 几何形体程序中，需要写 "点" 类，也需要写 "圆" 类，两者的关系就是复合关系
- 每一个 "圆" 对象里都包含（有）一个 "点" 对象，这个 "点" 对象就是圆心

```
class CPoint{
    double x, y;
    friend class CCircle;  //便于CCirle类操作其圆心
};
class CCircle{
    double r;
    CPoint center;
};
```

复合关系的使用

- 写一个小区养狗管理程序
  - 需要写一个"业主"类，还需要写一个"狗"类
  - 而狗是有"主人"的，主人当然是业主

    (假定狗只有一个主人，但一个业主可以有最多10条狗)

复合关系的使用


```
class CDog;
class CMaster {
    CDog dogs[10];
};
class CDog {
    CMaster m;
};
```

- 这段程序有问题吗？

复合关系的使用


```
class CDog;
class CMaster {
    CDog dogs[10]; //无法编译，CDog对象无定义
};
class CDog {
    CMaster m;
};
```

- 避免循环定义的方式：在一个类中使用另一个类的指针，而不是对象作为成员变量

复合关系的使用

- **正确写法：**
  - 为 "狗" 类设一个 "业主" 类的成员对象指针
  - 为 "业主" 类设一个 "狗" 类的对象指针数组

```
class CDog;
class CMaster {
    CDog * dogs[10];
};
class CDog {
    CMaster * m;
};
```

继　承

- 派生类可以定义一个**和基类成员同名**的成员，这叫**隐藏***
- 在派生类中访问这类成员时，缺省的情况是访问派生类中定义的成员
- 要在派生类中访问由**基类定义的同名成员**时，要使用**作用域符号::**

* 也有称为**覆盖**的

基类和派生类有同名成员的情况


```
class base {
    int j;
public:
    int i;
    void func();
};
```

```
class derived : public base {
public:
    int i;
    void access();
    void func();
};
```

```
void derived::access(){
    j = 5;              //error
    i = 5;              //派生类的 i
    base::i = 5;        //基类的 i
    func();             //派生类的
    base::func();       //基类的
}
derived  obj;
obj.i = 1;
obj.base::i = 1;
```

Obj占用的存储空间

一般地，基类和派生类不定义同名成员变量

测一测

- **以下哪种派生关系是合理的：**

A. 从“虫子”类派生出“飞虫”类

B. 从“点”类派生出“圆”类

C. 从“狼”类派生出“狗”类

D. 从“爬行动物”类派生出“哺乳动物”类

测一测

- 以下哪种派生关系是合理的：

A. 从“虫子”类派生出“飞虫”类

B. 从“点”类派生出“圆”类

C. 从“狼”类派生出“狗”类

D. 从“爬行动物”类派生出“哺乳动物”类

测一测

- **派生类和基类有同名同参数表的成员函数，这种现象：**

A. 叫重复定义，是不允许的

B. 叫函数的重载

C. 叫隐藏，在派生类中基类的同名函数就没用了

D. 叫隐藏，体现了派生类对从基类继承得到的特点的修改

测一测

- **派生类和基类有同名同参数表的成员函数，这种现象：**

A. 叫重复定义，是不允许的

B. 叫函数的重载

C. 叫隐藏，在派生类中基类的同名函数就没用了

D. **叫隐藏，体现了派生类对从基类继承得到的特点的修改**

测一测

- **以下说法正确的是：**

A. 派生类可以和基类有同名成员函数，但是不能有同名成员变量

B. 派生类成员函数中，可以调用基类的同名同参数表的成员函数

C. 派生类和基类的同名成员函数必须参数表不同，否则就是重复定义

D. 派生类和基类的同名成员变量存放在相同的存储空间

测一测

- **以下说法正确的是：**

A. 派生类可以和基类有同名成员函数，但是不能有同名成员变量

B. **派生类成员函数中，可以调用基类的同名同参数表的成员函数**

C. 派生类和基类的同名成员函数必须参数表不同，否则就是重复定义

D. 派生类和基类的同名成员变量存放在相同的存储空间

目　录

- 基本概念：继承，基类，派生类
- 派生类的成员组成，可见性
- 派生类的构造，析构
- 派生类与基类的指针类型转换

访问权限说明符：protected

- 基类的**private成员**：可以被下列函数访问
  - 基类的成员函数
  - 基类的友员函数
- 基类的**public成员**：可以被下列函数访问
  - 基类的成员函数
  - 基类的友员函数
  - 派生类的成员函数
  - 派生类的友员函数
  - 其他的函数

访问权限说明符：protected

- 基类的**protected成员**：可以被下列函数访问
- 可访问范围比private成员大，比public成员小
  - 基类的成员函数
  - 基类的友员函数
  - **派生类的成员函数**可以访问**当前对象**的基类的保护成员

    → **this指针指向的对象**

    → 也可以访问**其它同类对象**的基类的保护成员
  - 派生类的友元函数也可以访问基类的保护成员

保护成员


```
class Father {
    private:   int nPrivate;      //私有成员
    public:    int nPublic;       //公有成员
    protected: int nProtected;    // 保护成员
};
class Son : public Father{
    void AccessFather () {
        nPublic = 1;          // OK
        nPrivate = 1;         // wrong
        nProtected = 1;       // OK，访问从基类继承的protected成员
        Son s;
        s.nProtected = 1;     //OK，访问其它同类对象的基类保护成员
        Father f;
        f.nProtected = 1;     //wrong，f不是函数所作用的当前对象
    }
};
```

保护成员


```
class Father {
    private:   int nPrivate;      // 卧室（绝对私密空间）
    public:    int nPublic;       // 客厅（任何人都可以进入）
    protected: int nProtected;    // 书房（只对家人开放）
};
class Son : public Father{
    void AccessFather () {
        nPublic = 1;          // ☑ 客厅当然可以进
        nPrivate = 1;         // ✕ 父母的卧室不能随便进!
        nProtected = 1;       // ☑ 书房是家人共享的
        Son s;                // 另一个儿子（兄弟）的房子
        s.nProtected = 1;     // ☑ 兄弟家的书房也可以用
        Father f;             // 隔壁老王的房子（不是自己家）
        f.nProtected = 1;     // ✕ 不能去别人家使用书房!
    }
};
```

保护成员


```
class Father {
    private:   int nPrivate;     // 卧室（绝对私密空间）
    public:    int nPublic;      // 客厅（任何人都可以进入）
    protected: int nProtected;   // 书房（只对家人开放）
};
class S
    voi

    }
};
```

访问位置	卧室(private)	书房(protected)	客厅(public)
Father自己	✅	✅	✅
Son（在自己家）	❌	✅	✅
Son（在兄弟家）	❌	✅	✅
Son（在别人家）	❌	❌	✅

保护成员


```
class Father {
    private:   int nPrivate;     //私有成员
    public:    int nPublic;      //公有成员
    protected: int nProtected;   // 保护成员
};
int main() {
    Father f;
    Son s;
    f.nPublic = 1;       // OK
    s.nPublic = 1;       // OK
    f.nProtected = 1;    // error
    f.nPrivate = 1;      // error
    s.nProtected = 1;    // error
    s.nPrivate = 1;      // error
    return 0;
}
```

三种继承方式

- **公有继承（public）**
  - 基类的公有成员和保护成员作为派生类的成员时，它们都保持**原有的状态**，而基类的私有成员**仍然是私有的**，不能被这个派生类的子类所访问
- **私有继承（private）**
  - 基类的公有成员和保护成员都作为派生类的**私有成员**，并且不能被这个派生类的子类所访问
- **保护继承（protected）**
  - 基类的所有公有成员和保护成员都成为派生类的**保护成员**，并且只能被它的派生类成员函数或友元访问，基类的私有成员仍然是私有的

三种继承方式

- 公有继承（public）
- 私有继承（private）
- 保护继承（protected）

	public	protected	private
公有继承	public	protected	不可见
私有继承	private	private	不可见
保护继承	protected	protected	不可见

public继承


```
class base {
private:
    int m;
public:
    int q;
    base(int i=0):m(i){ }
};
class derived: public base {
public:
    int n;
    derived(int j=0):base(j){}
};
base b;
derived d;
```

***base* 类**

```
private
    int m;
public:
    int q;
    … …
```

派生

***derived* 类**

*base*部分

```
private:
    int m;
public:
    int q;
    … …
```

*derived*部分

```
public:
    int n;
    … …
```

目　录

- 基本概念：继承，基类，派生类
- 派生类的成员组成，可见性
- 派生类的构造，析构
- 派生类与基类的指针类型转换

派生类的构造函数


```
class Bug  {
private :
    int nLegs;
    int nColor;
public:
    int nType;
    Bug (int legs,  int color);
    void PrintBug(){ }
};
class FlyBug: public Bug // FlyBug是Bug的派生类
{
    int nWings;
public:
    FlyBug(int legs,  int color,  int wings);
};
```

派生类的构造函数


```
Bug::Bug(int legs,  int color){
    nLegs = legs;
    nColor = color;
}
//错误的FlyBug构造函数
FlyBug::FlyBug (int legs,  int color,  int wings){
    nLegs = legs;      // 不能访问
    nColor = color;  // 不能访问
    nType = 1; // ok
    nWings = wings;
}
//正确的FlyBug构造函数
FlyBug::FlyBug (int legs,  int color,  int wings):Bug(legs,  color){
    nWings = wings;
}
```

派生类的构造函数


```
int main() {
    FlyBug fb (2, 3, 4);
    fb.PrintBug();
    fb.nType = 1;
    fb.nLegs = 2 ;  // error!  nLegs is private
    return 0;
}
```

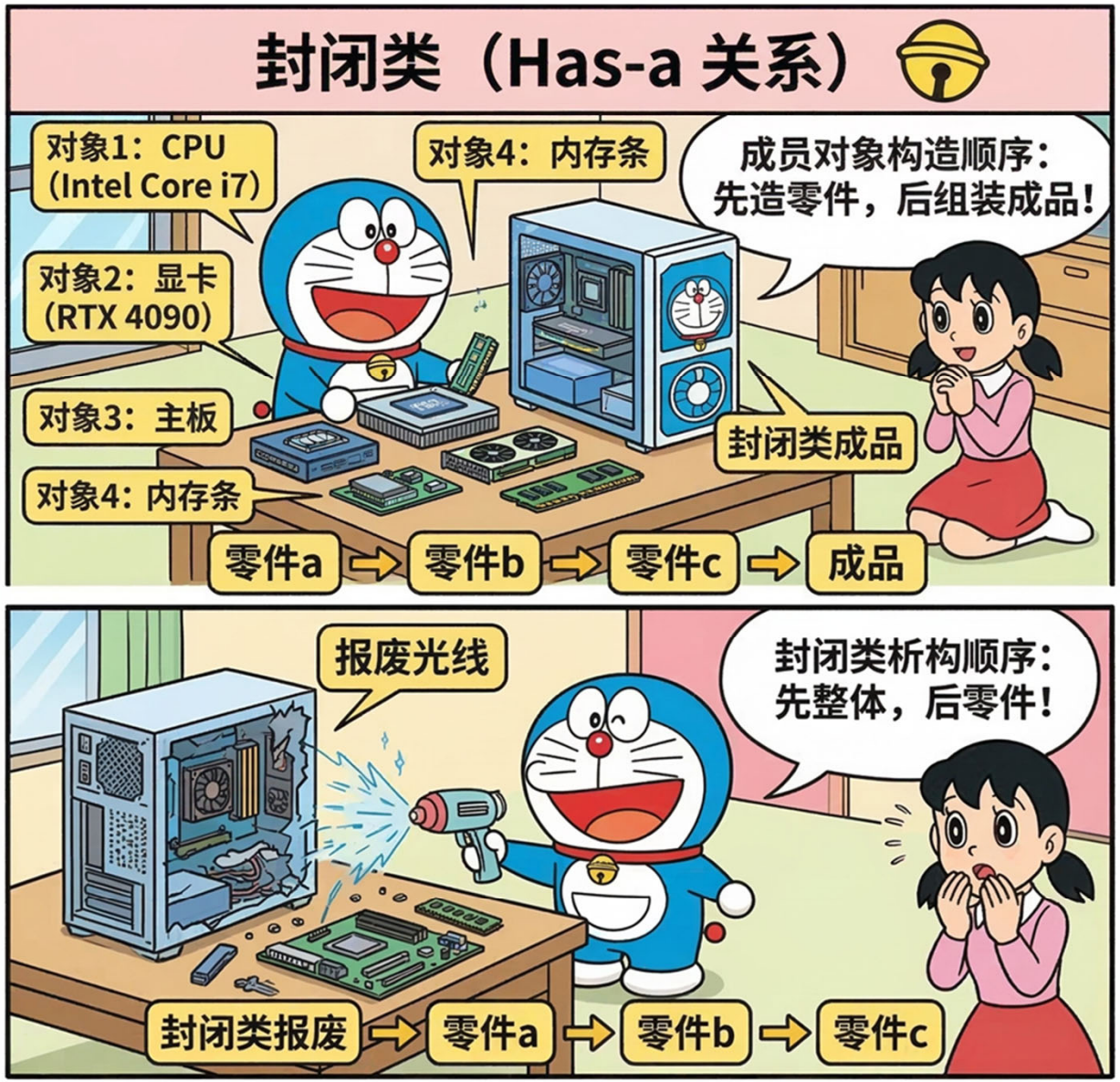
派生类的构造函数

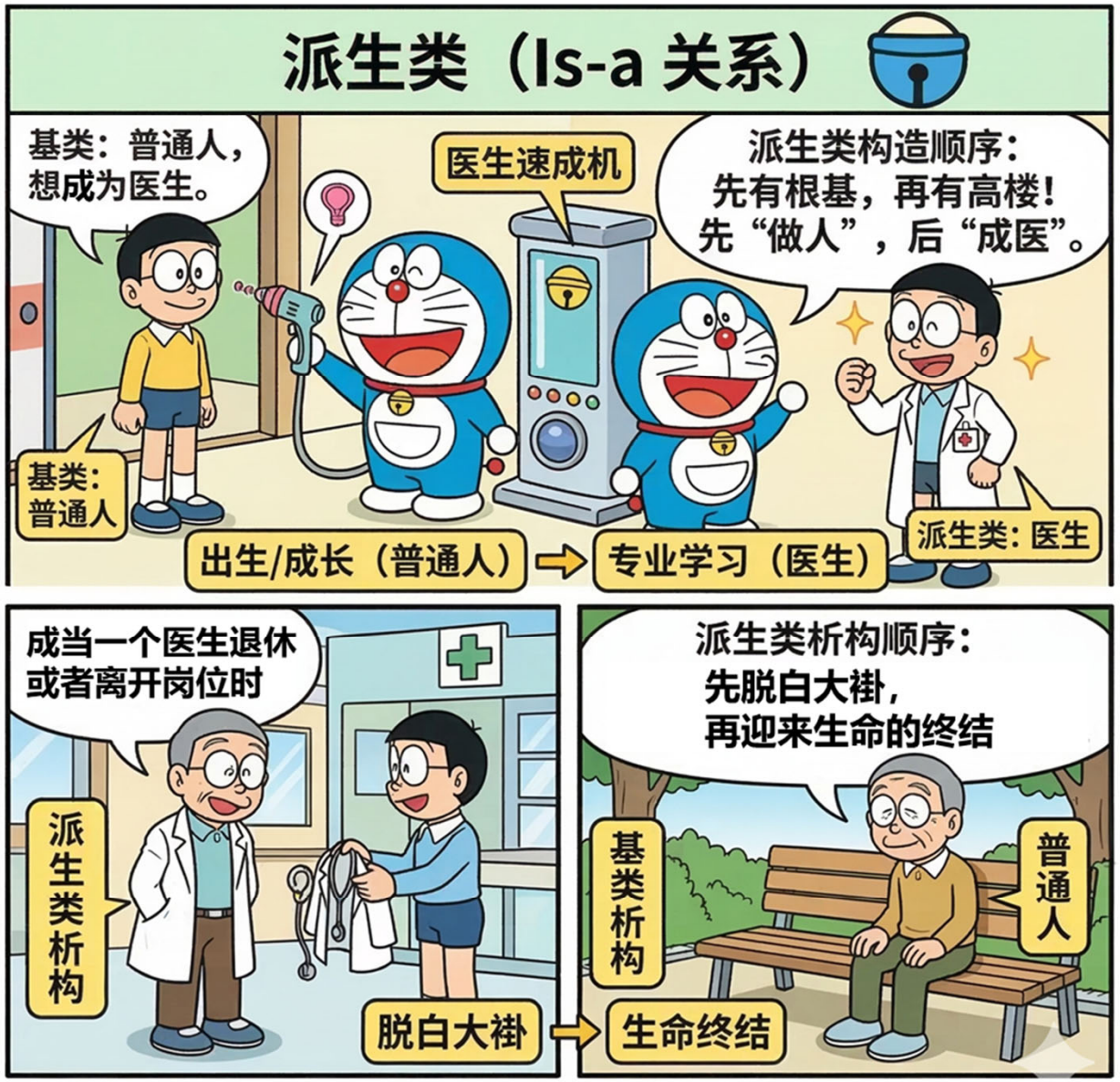

```
FlyBug fb (2, 3, 4);
```

- 在创建**派生类的对象**时，需要调用**基类的构造函数**
  - 初始化派生类对象中从基类继承的成员
  - 在执行**派生类的构造函数之前**，总是先执行**基类的构造函数**
- **调用基类构造函数的两种方式**
  - 显式方式：在派生类的构造函数中，为基类构造函数提供参数

    `derived::derived(arg_derived-list):base(arg_base-list)`
  - 隐式方式：在派生类的构造函数中，省略基类构造函数时，派生类的构造函数则自动调用基类的默认构造函数
- **派生类的析构函数被执行时，执行完派生类的析构函数后，自动调用基类的析构函数**

哆啦A梦的C++魔法：类和对象的秘密顺序！





封闭类先零件，派生类先基类；析构顺序反过来！

派生类的构造函数


```
class Base {
public:
    int n;
    Base(int i):n(i)
    { cout << "Base " << n << " constructed" << endl; }
    ~Base()
    { cout << "Base " << n << " destructed" << endl;  }
};
class Derived:public Base  {
public:
    Derived(int i):Base(i)
    {  cout << "Derived constructed" << endl;  }
    ~Derived()
    {  cout << "Derived destructed" << endl;  }
};
```

派生类的构造函数


```
int main()
{
    Derived Obj(3);
    return 0;
}
```

```
Base 3 constructed
Derived constructed
Derived destructed
Base 3 destructed
```

包含成员对象的派生类的构造函数


```
class Skill{
public:
    Skill(int n) { }
};
class FlyBug: public Bug {
    int nWings;
    Skill  sk1,  sk2;
public:
    FlyBug( int legs,  int color,  int wings);
};

FlyBug::FlyBug( int legs,  int color,  int wings):
    Bug(legs,  color),  sk1(5),  sk2(color)  {
    nWings = wings;
}
```

包含成员对象的派生类的构造函数

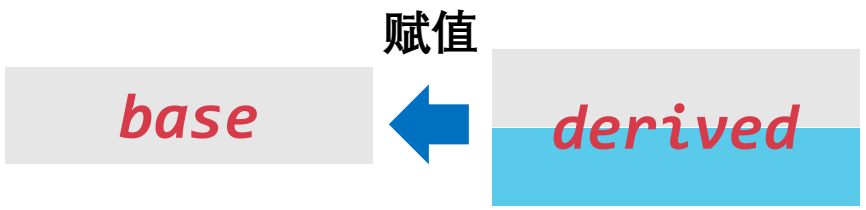
- **创建派生类的对象时，执行一个派生类的构造函数之前：**
  - 调用**基类**的构造函数

    初始化派生类对象中从基类继承的成员
  - 调用**成员对象类**的构造函数

    初始化派生类对象中成员对象
- **派生类析构函数被执行时，执行完派生类的析构函数后：**
  - 调用**成员对象类**的析构函数
  - 调用**基类**的析构函数
- **析构函数的调用顺序与构造函数的调用顺序相反**

public继承的赋值兼容规则


```
class base { };
class derived : public base { };
base b;
derived d;
```



- 派生类的对象可以赋值给基类对象
  - `b = d;`
- 派生类对象可以初始化基类引用
  - `base & br = d;`
- 派生类对象的地址可以赋值给基类指针
  - `base * pb = & d;`
- 如果派生方式是 private 或 protected，则上述三条不可行

该对象不包含派生类定义的成员，因为没有派生类成员的存储空间（派生类部分在赋值时被“切掉”）
*C++ Primer P489

测一测

- **以下说法正确的是：**

A. 派生类对象生成时，派生类的构造函数先于基类的构造函数执行

B. 派生类对象消亡时，基类的析构函数先于派生类的析构函数执行

C. 如果基类有无参构造函数，则派生类的构造函数就可以不带初始化列表

D. 在派生类的构造函数中不可以访问基类的成员变量

测一测

- **以下说法正确的是：**

A. 派生类对象生成时，派生类的构造函数先于基类的构造函数执行

B. 派生类对象消亡时，基类的析构函数先于派生类的析构函数执行

C. 如果基类有无参构造函数，则派生类的构造函数就可以不带初始化列表

D. 在派生类的构造函数中不可以访问基类的成员变量

目　录

- 基本概念：继承，基类，派生类
- 派生类的成员组成，可见性
- 派生类的构造，析构
- 派生类与基类的指针类型转换

基类与派生类的指针强制转换

- 公有派生的情况下，
- 派生类对象的地址可以直接赋值给基类指针

```
Base * ptrBase = & objDerived;
```

  - ptrBase指向的是一个Derived类的对象
  - *ptrBase可以看作一个Base类的对象，访问它的public成员
  - 直接通过ptrBase，不能够访问objDerived由Derived类扩展的成员
- 即便基类指针指向的是一个派生类的对象，也不能通过基类指针访问基类没有，而派生类中有的成员

基类与派生类的指针强制转换

- 通过强制指针类型转换

  可以把ptrBase转换成Derived类的指针

```
Base * ptrBase = & objDerived;
Derived *ptrDerived = (Derived * ) ptrBase;
```

  - 程序员要保证ptrBase指向的是一个Derived类的对象，否则很容易会出错

基类与派生类的指针强制转换


```
class Base {
protected:
    int n;
public:
    Base(int i):n(i){
        cout << "Base " << n << " constructed" << endl;
    }
    ~Base() {
        cout << "Base " << n << " destructed" << endl;
    }
    void Print() { cout << "Base:n=" << n << endl; }
};
```

基类与派生类的指针强制转换


```
class Derived:public Base  {
public:
    int v;
    Derived(int i):Base(i),  v(2 * i) {
        cout << "Derived constructed" << endl;
    }
    ~Derived()  {
        cout << "Derived destructed" << endl;
    }
    void Func() { } ;
    void Print() {
        cout << "Derived:v=" << v << endl;
        cout << "Derived:n=" << n << endl;
    }
};
```

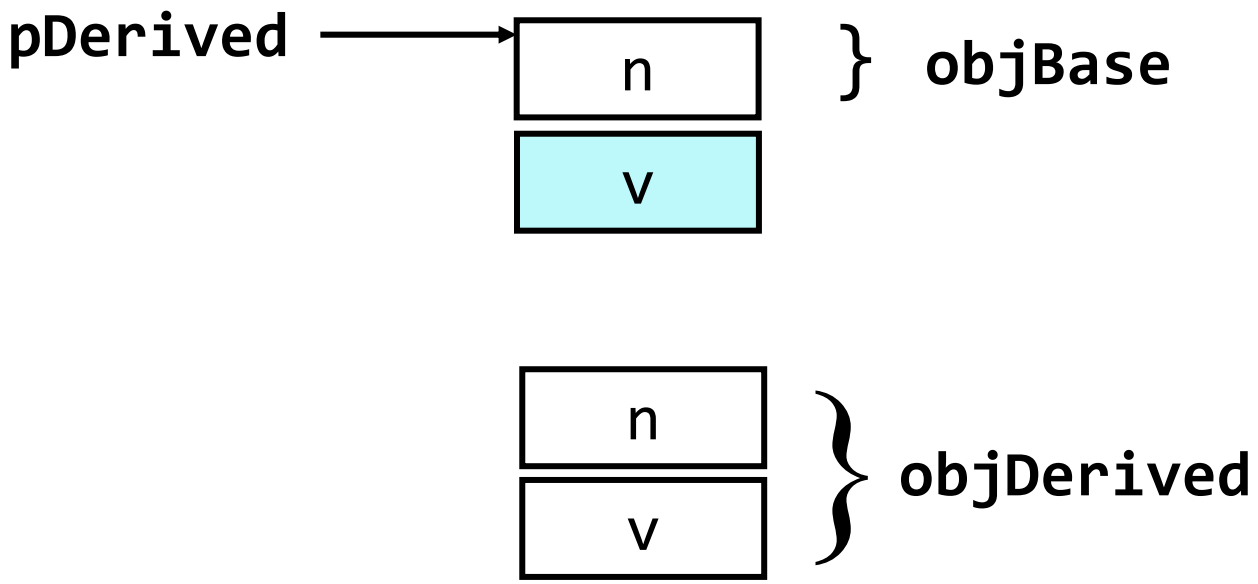
基类与派生类的指针强制转换


```
int main()  {
    Base objBase(5);
    Derived objDerived(3);
    Base * pBase = & objDerived ;
    //pBase->Func();        //err; Base类没有Func()成员函数
    //pBase->v = 5;         //err; Base类没有v成员变量
    pBase->Print();         //调用基类函数Print()
    //Derived * pDerived = & objBase;    //error
    Derived * pDerived = (Derived *)(& objBase);
    pDerived->Print();      //慎用，可能出现不可预期的错误
    pDerived->v = 128;      //往别人的空间里写入数据
                            //→ 可能引起问题
    objDerived.Print();
    return 0;
}
```

根据指针/引用类型来决定调用的函数

基类与派生类的指针强制转换


```
Derived * pDerived = (Derived *)(& objBase);
```



基类与派生类的指针强制转换


```
int main()  {
    Base objBase(5);
    Derived objDerived(3);
    Base * pBase = & objDerived ;
    //pBase->Func();        //err;
    //pBase->v = 5;         //err;
    pBase->Print();         //调用基类函数Print()
    //Derived * pDerived = & objBase;    //error
    Derived * pDerived = (Derived *)(& objBase);
    pDerived->Print();      //慎用,
    pDerived->v = 128;      //往别人的空间里写入数据
                            //→ 可能引起问题
    objDerived.Print();
    return 0;
}
```

```
Base 5 constructed
Base 3 constructed
Derived constructed
Base:n=3
Derived:v=6422060
Derived:n=5
Derived:v=6
Derived:n=3
Derived destructed
Base 3 destructed
Base 5 destructed
```

直接基类与间接基类

- 类A派生类B，类B可再派生类C，类C派生类D，...
  - 类A是类B的直接基类
  - 类B是类C的直接基类，类A是类C的间接基类
  - 类C是类D的直接基类，类A，B是类D的间接基类
- 在声明派生类时，派生类的首部只需要列出它的**直接基类**
  - 派生类的首部**不要列出它的间接基类**
  - 派生类沿着类的层次自动向上继承它的间接基类
  - 派生类的成员包括
    - 派生类自己定义的成员
    - 直接基类中定义的成员
    - 间接基类的全部成员

直接基类与间接基类


```
class Base {
public:
    int n;
    Base(int i):n(i)  {
        cout << "Base " << n << " constructed" << endl;
    }
    ~Base() {
        cout << "Base " << n << " destructed" << endl;
    }
};    //基类
```

直接基类与间接基类


```
class Derived: public Base {
public:
    Derived(int i):Base(i) {
        cout << "Derived constructed" << endl;
    }
    ~Derived() {
        cout << "Derived destructed" << endl;
    }
};
```

直接基类与间接基类


```
class MoreDerived: public Derived {
public:
    MoreDerived():Derived(4) {
        cout << "More Derived constructed" << endl;
    }
    ~MoreDerived() {
        cout << "More Derived destructed" << endl;
    }
};
int main() {
    MoreDerived Obj;
    return 0;
}
```

直接基类与间接基类


```
class MoreDerived: public Derived {
public:
    MoreDerived():Derived(4) {
        cout << "More Derived constructed" << endl;
    }
    ~MoreDerived() {
        cout << "More Derived destructed" << endl;
    }
};
int main() {
    MoreDerived Obj;
    return 0;
}
```

```
Base 4 constructed
Derived constructed
More Derived constructed
More Derived destructed
Derived destructed
Base 4 destructed
```

多继承（考试不要求）

- 一个类可以从多个基类派生而来，以继承多个基类的成员–这种派生称作 **"多重继承"**

```
class derived:access-specifier1 base1, access-specifier2 base2, ...  {
    ...
};
```

- access-specifier$_i$可以是private，protected，public之一

多继承的派生类构造函数


```
class base1 {
    int i;
public:
    base1(int n) { i = n; }
};
class base2 {
    int j;
public:
    base2(int n) { j = n; }
};
class derived : public base1, public base2 {
public:
    derived( int x );
};
derived::derived( int x ): base1(x), base2(0) { }
```

多继承的派生类构造函数

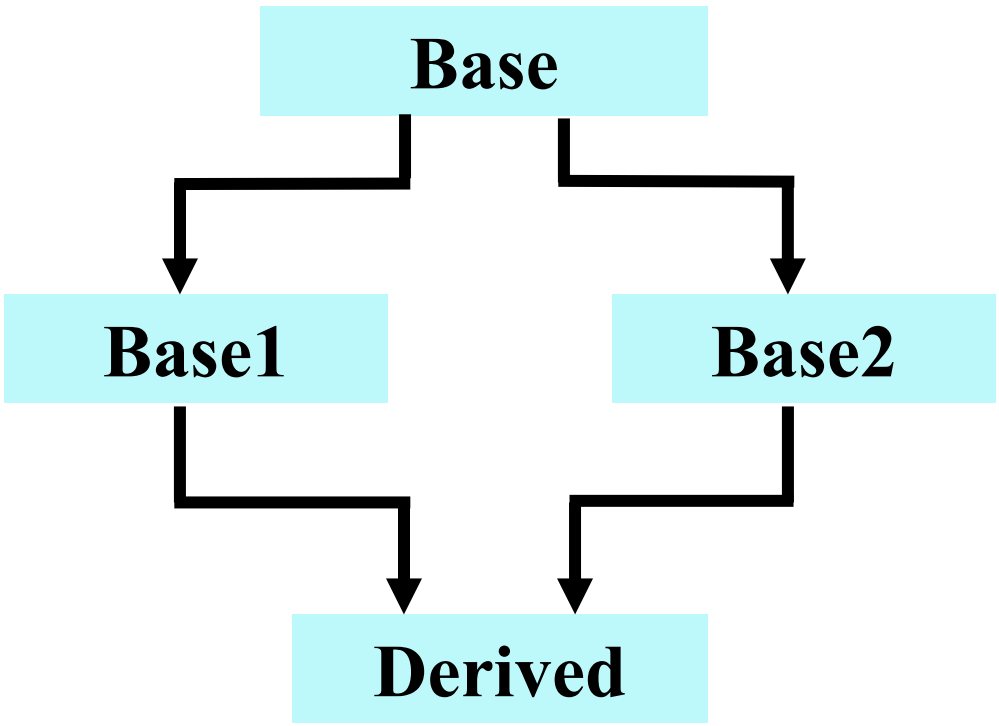
- 多重继承中，派生类对象创建时，先**按继承顺序调用基类的构造函数**，然后再调用派生类的构造函数
- 如果派生类是封闭类，那么**成员对象的构造函数在基类的构造函数调用结束后依次调用**，最后才调用派生类的构造函数
- 多重继承中，派生类对象的创建过程
  - 按继承顺序调用基类的构造函数
  - 依次调用成员对象的构造函数
  - 调用派生类的构造函数

多继承中基类的构造函数调用


```
class Base {
public:
    int val;
    Base() { cout << "Base Constructor" << endl; }
    ~Base() {
        cout << "Base Destructor" << endl;
    }
};
class Base1: public Base {  };
class Base2: public Base {  };
class Derived: public Base1, public Base2 {   };
```

```
int main() {
    Derived d;
}
```

```
Base Constructor
Base Constructor
Base Destructor
Base Destructor
```



基类Base的构造函数和析构函数都被调用两次

多重继承的二义性


```
class base1 {
private:
    int b1;
    void set(int i) {  b1 = 1; }
public:
    int i;
};
class base2 {
private:
    int b2;
public:
    void set(int i)  { b2 = i; }
    int get() {  return b2; }
    int i;
};
```

多重继承的二义性


```
class derived:  public base1, public base2{
public:
    void print() {
        printf( "%d", get() );
        set(5);             // 二义性, error
        base2::set(5) ;     // ok
        base1::set(5);      // error, set is private in base1
    }
};
```

derived对象的属性

int base1:: private b1
int base1:: public i
int base2:: private b2
int base2:: public i

derived对象的操作

void base1:: private set(int i)
void base2:: public set(int i)
int base2:: public get(int i)
public void print()

多重继承的二义性


```
int main ()  {
    derived d;
    d.set(10);          //二义性
    d.base1::set(10);   //error, 不能访问私有成员
    d.base2::set(5);
    d.base1::i =5;
    d.base2::i = 5;
    Return 0;
}
```

二义性检查在访问权限检查之前进行，不能靠成员的访问权限来消除二义性

```
class A {    public: void fun();    }
class B {    private: void fun();    }
class C : public A, public B {    };
C obj;
Obj.fun(); // 二义性
```

总　　结

- **基本概念：继承，基类，派生类**
  - 合理派生
- **派生类的成员组成，可见性**
  - private/protected成员的继承性
  - 派生类的成员函数 – 不能访问基类中的private成员
  - 派生类的成员函数 – 访问基类的protected成员
- **派生类的构造，析构**
  - 构造顺序：基类，对象成员，派生
  - 析构反之
- **派生类与基类的指针类型转换**
  - f(派生类)→y(基类对象)：f：对象/对象地址；y：对象/引用/指针
  - 基类指针→(强制指针类型转换)派生类指针